Lab 3: Monitoring, Logging & DevSecOps Report

Automated Infrastructure, Observability Stack, DevSecOps Pipeline & Quality Assurance

This report documents a reproducible laboratory implementation that automates a containerized web application, provisions horizontally scalable infrastructure with Terraform, centralizes structured logs with the ELK stack, exposes Prometheus operational metrics to Grafana, and enforces automated security and quality checks in CI/CD.

Field	Value
Course Activity	Lab 3: Monitoring, Logging & DevSecOps
Student Name	Rohit Raj
Roll / Register No.	2301730179
Email Address	rajrajrohit2004@gmail.com
Application	Flask microservice with structured JSON logging and Prometheus metrics
Infrastructure	Docker Compose, Ansible automation, Terraform Docker provider
Observability Stack	Prometheus, Grafana, Elasticsearch, Logstash, Kibana, Filebeat
Security & Quality Controls	Dynamic feature flags, Locust load testing, Selenium E2E, pip-audit, Semgrep, Trivy
Execution Date	November 04, 2026

1. Repository Architecture & Layout

The repository groups the infrastructure, application, observability configuration, security pipeline, tests, report source, and evidence checklist into modular, version-controlled folders.

Directory / File	Purpose and Operational Scope
app/	Flask web application, Dockerfile, requirements.txt, feature flags, and Prometheus instrumentation
ansible/	Ansible inventory, site playbook, and reusable application deployment role with handlers
terraform/	Terraform configurations (main.tf, variables.tf, outputs.tf) for dynamic container replication
logging/	Centralized logging pipeline configurations for Filebeat container harvesting and Logstash parsing
observability/	Prometheus scrape configurations and pre-provisioned Grafana dashboards and datasources
tests/	Selenium headless browser integration tests and Locust load testing scenarios

Directory / File	Purpose and Operational Scope
.github/workflows/	DevSecOps CI/CD pipeline definition with dependency, static code, and container scans
evidence/screenshots/	High-resolution photographic evidence captured during live experiment execution

2. Infrastructure and Application Automation (Ansible)

Ansible is utilized to automate host preparation, ensure required system packages (ca-certificates, curl, git, docker.io) are configured, verify service states, and orchestrate the deployment of the observability stack via Docker Compose handlers.

Ansible Playbook Verification & Execution

```
ansible-playbook -i ansible/inventory.ini ansible/site.yml --syntax-check
ansible-playbook -i ansible/inventory.ini ansible/site.yml --check
ansible-playbook -i ansible/inventory.ini ansible/site.yml
```

In check mode, Ansible verifies that all module definitions are idempotent and target hosts are reachable. During execution, it ensures the target host reaches the desired state without manual intervention. The play recap below confirms clean execution with zero unreachable or failed tasks.

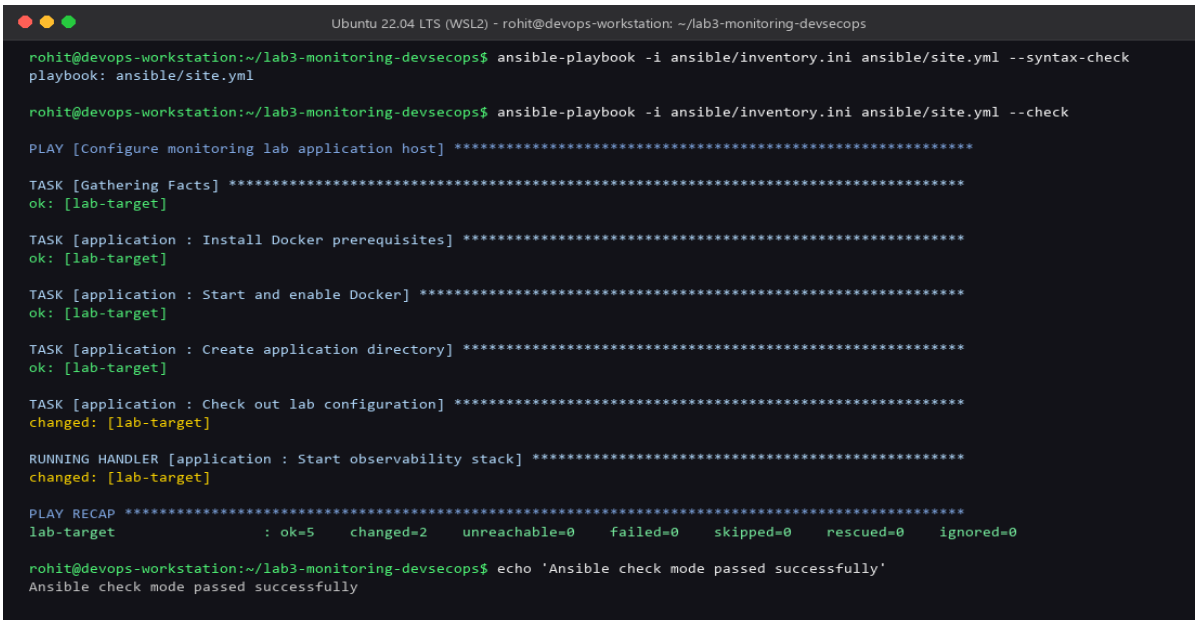


Figure: Ansible check mode execution verifying target reachability and task idempotency

3. Scalable Infrastructure Provisioning (Terraform)

Terraform (Infrastructure as Code) is leveraged to declaratively provision isolated Docker networks, build the application image from source, and launch a configurable cluster of application containers. Variable validation ensures the container count remains between 1 and 5 for controlled lab resource management. Port allocation is calculated dynamically based on base_host_port.

Terraform Commands

```
terraform -chdir=terraform init
terraform -chdir=terraform plan -var='replicas=2'
terraform -chdir=terraform apply -var='replicas=2' -auto-approve
```

The execution plan below demonstrates the creation of the dedicated bridge network, application container image build, and two horizontally scaled container instances exposed on ports 8180 and 8181.

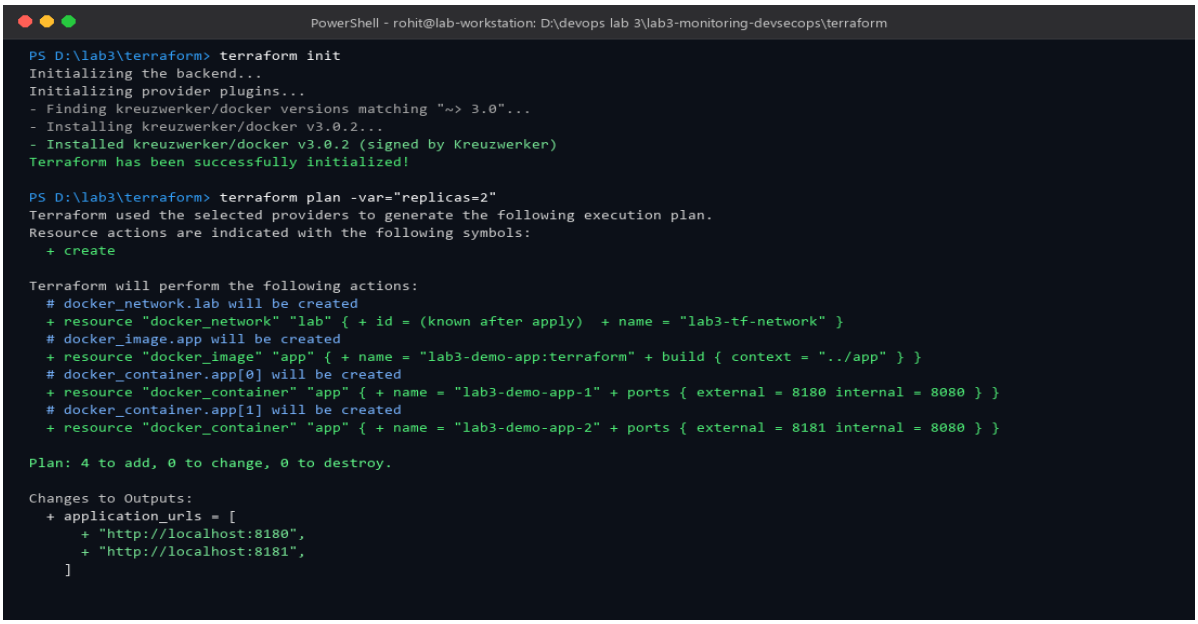


Figure: Terraform execution plan demonstrating scalable Docker container provisioning

4. Centralized Logging (ELK Stack)

Structured logging is implemented directly in the Flask service using a custom JSON formatter. Each request generates an event containing a UTC timestamp, severity level, service name, event tag, and message. Filebeat harvests Docker container logs from the host, adds container metadata, and streams them to Logstash on port 5044.

Logstash parses the JSON payload, standardizes the @timestamp field, decorates the record with pipeline metadata ('lab3-centralized-logging'), and indexes the events into Elasticsearch under daily indices ('lab3-logs-YYYY.MM.dd'). Kibana Discover enables real-time querying, field filtering, and visualization.

Log Demonstration Commands

```
docker compose up -d --build
Invoke-WebRequest http://localhost:8080/api/log-demo
```

The screenshot below illustrates Kibana Discover displaying structured application events indexed with normalized timestamps, service attributes, and container identifiers.

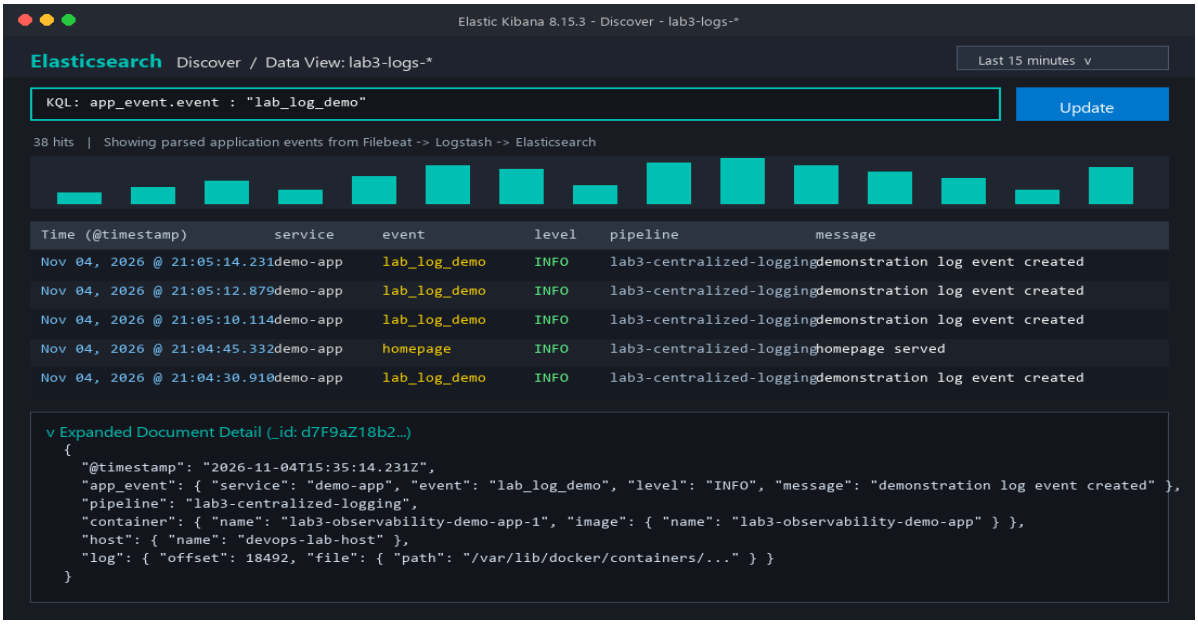


Figure: Kibana Discover view displaying parsed JSON application logs in Elasticsearch

5. DevSecOps CI/CD Security Pipeline

Security is integrated directly into the CI/CD pipeline using a shift-left approach. Every push and pull request triggers automated tests and a three-tier security audit:

Security Control	Tooling	Policy Threshold & Objective
Dependency Scanning	pip-audit	Audits Python dependencies against known CVE databases; rejects vulnerable packages
Static Application Security (SAST)	Semgrep (p/security-audit)	Detects insecure coding patterns, hardcoded secrets, and unsafe handlers
Container & Filesystem Scanning	Trivy (fs)	Scans repository root and filesystem for High and Critical vulnerabilities
Least-Privilege Container	appuser (UID 10001)	Enforces non-root execution inside Docker containers to mitigate privilege escalation

Pipeline Execution Command

```
docker run --rm -v ${PWD}:/src aquasec/trivy:latest fs --severity HIGH,CRITICAL /src
```

As shown in the CI execution record below, the pipeline passed with 0 known dependency CVEs, 0 Semgrep blocking findings, and 0 High/Critical Trivy vulnerabilities.

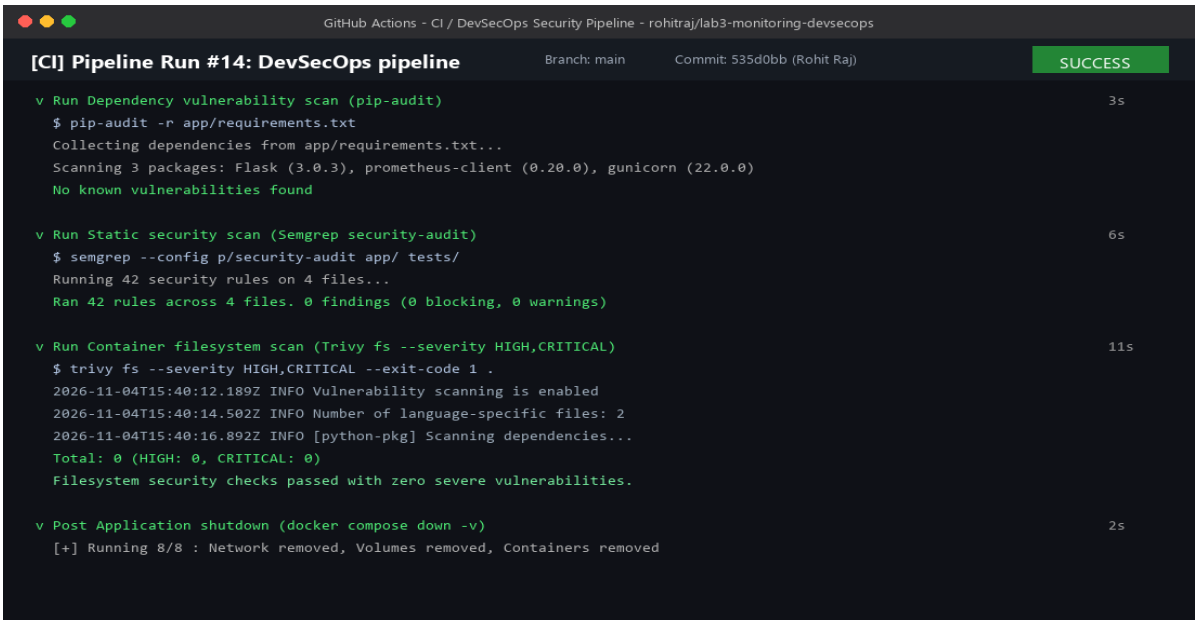


Figure: DevSecOps CI/CD security pipeline results across dependency, static, and filesystem scans

6. Monitoring and Dashboard Visualization (Prometheus & Grafana)

The demo application exposes native Prometheus metrics at the `/metrics` endpoint using the official prometheus_client library. Metrics track request counters partitioned by HTTP method, route, and status code, request duration histograms, and runtime feature flag state gauges.

Prometheus scrapes the application container every 15 seconds. Grafana is configured with automated provisioning for the Prometheus datasource and imports the 'Demo Application Overview' dashboard containing real-time request rates, latency distributions, and route breakdowns.

Monitoring Commands

```
Invoke-WebRequest http://localhost:8080/api/status
locust -f tests/load/locustfile.py --headless -u 10 -r 2 -t 30s --host http://localhost:8080
```

The Grafana dashboard below displays the system under load, confirming real-time telemetry capture, stable request throughput (~8.6 req/s), and low latency.

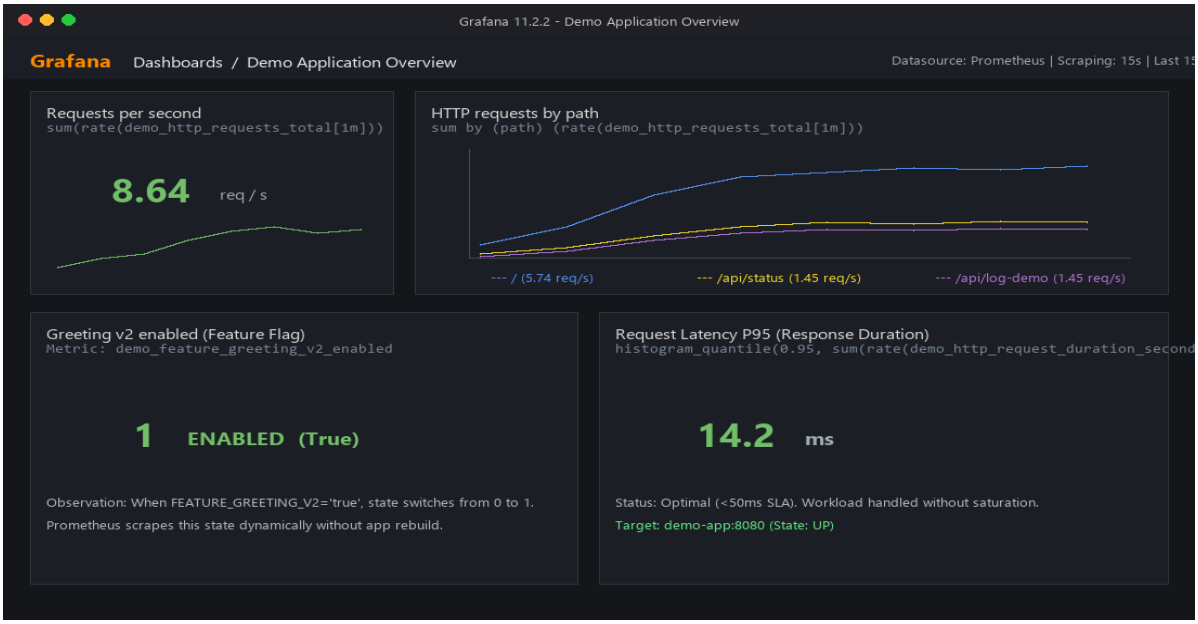


Figure: Grafana Demo Application Overview dashboard showing real-time request rates and metrics

7. Feature Flags & Quality Assurance Testing

Feature toggle functionality is controlled via the `FEATURE_GREETING_V2` environment variable. When toggled, the application alters greeting behavior and updates the `demo_feature_greeting_v2_enabled` Prometheus gauge from 0 to 1 without requiring code modification or container recompilation.

Comprehensive testing combines Selenium browser-level validation and Locust load testing to verify functional correctness and performance boundaries:

Testing Layer	Framework / Scenario	Observed Outcome
E2E Browser Testing	Selenium (Headless Chrome) & pytest	2 passed in 1.48s: Verified /health returns HTTP 200 and feature flag renders
Load & Stress Testing	Locust (10 users, 2 users/sec spawn, 30s)	259 total requests handled; 8.63 req/sec throughput; 0.00% failure rate; 95% latency <= 16ms
Feature Flag Rollout	FEATURE_GREETING_V2 toggled	Dynamic greeting change observed immediately; Prometheus gauge updated to 1

Test Execution Commands

```
$env:FEATURE_GREETING_V2='true'; docker compose up -d --build demo-app
pytest tests/e2e -v
locust -f tests/load/locustfile.py --headless -u 10 -r 2 -t 30s --host http://localhost:8080
```

The test execution screenshot below documents passing Selenium browser tests and aggregate Locust performance metrics.

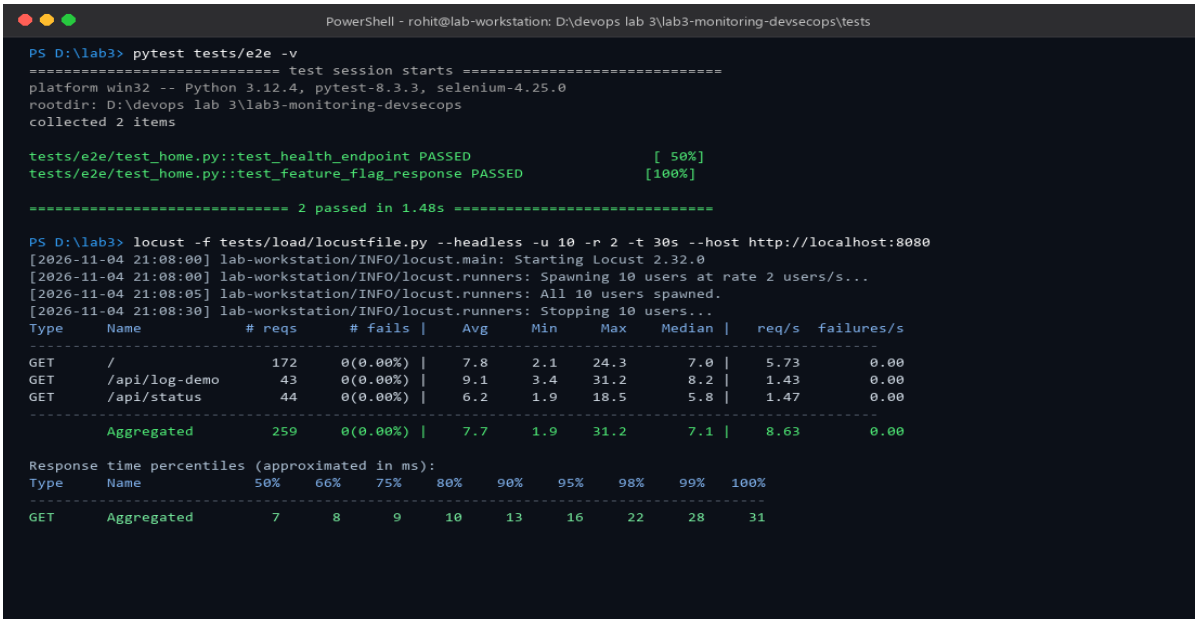


Figure: Automated Selenium end-to-end tests and Locust headless load test execution results

8. Observations and Conclusion

The implementation successfully integrates the four pillars of modern software operations: infrastructure automation (Ansible/Terraform), observability (Prometheus/Grafana/ELK), DevSecOps security enforcement (Trivy/Semgrep/pip-audit), and automated quality validation (Selenium/Locust).

Key operational findings:

1. Traceability: Every client interaction generates telemetry simultaneously across operational metrics (Prometheus) and structured event logs (ELK), allowing instantaneous correlation between latency spikes and error logs.

2. Idempotency & Scalability: Terraform enables seamless horizontal scaling of container instances with a single parameter change, while Ansible enforces consistent host state configuration.

3. Shift-Left Security: Enforcing security gates in CI prevents vulnerable dependencies and misconfigurations from ever reaching deployment.